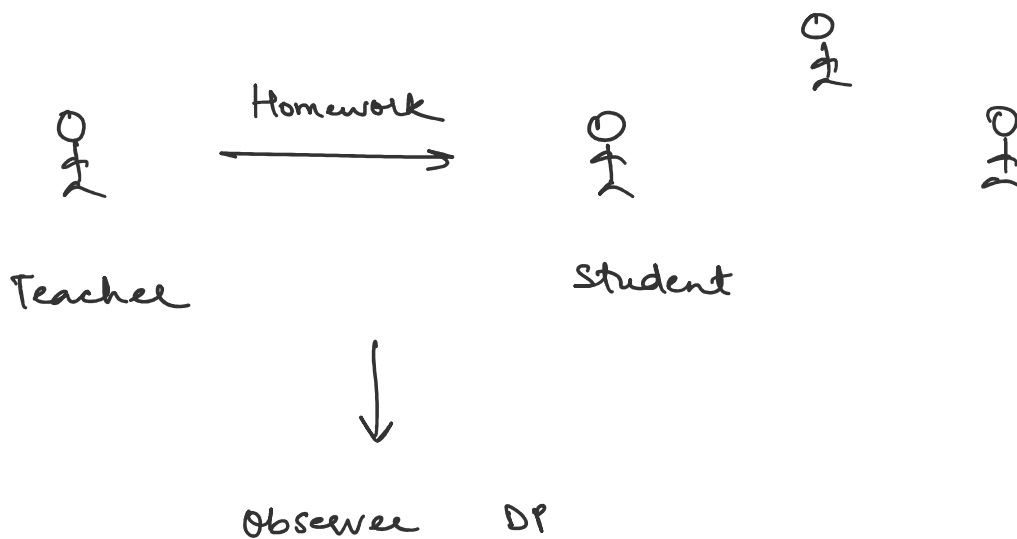
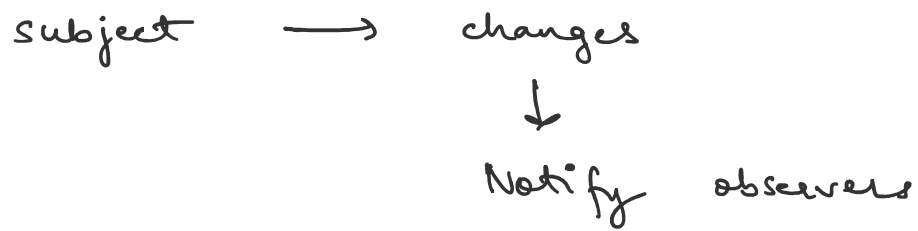


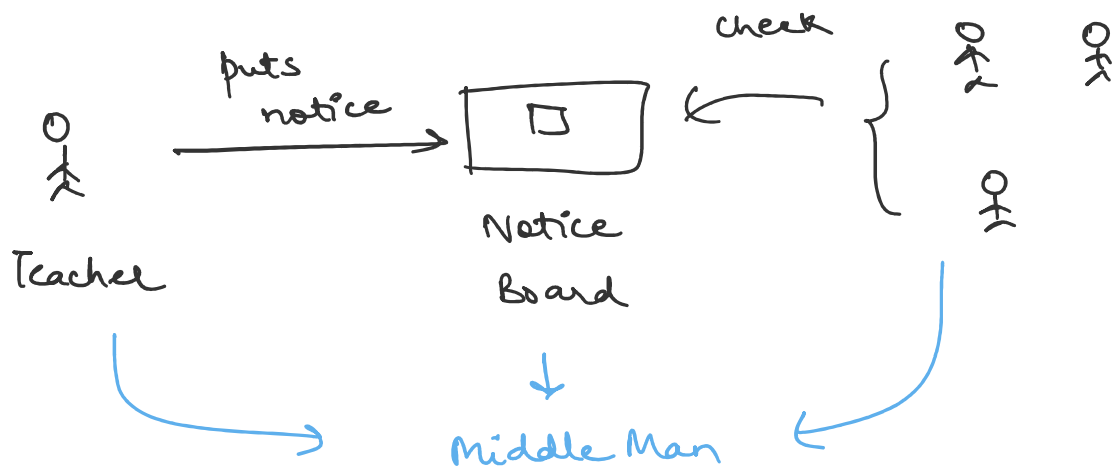
Class 17

Friday, July 17, 2026 9:06 PM

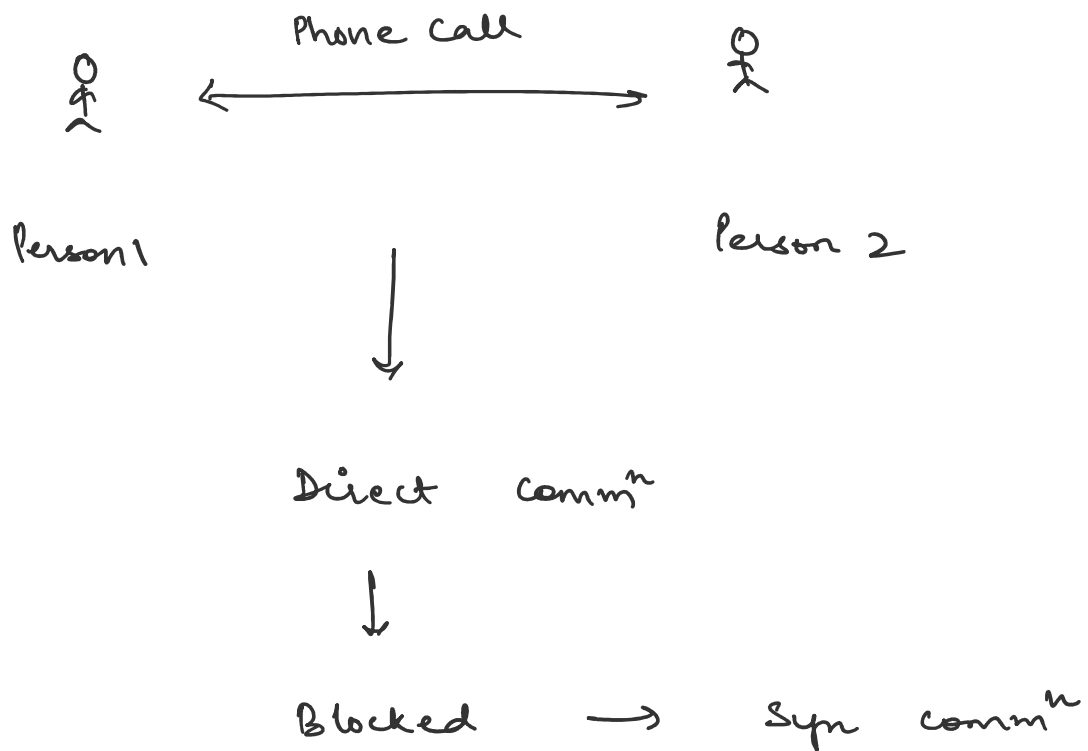
Observer Design Pattern

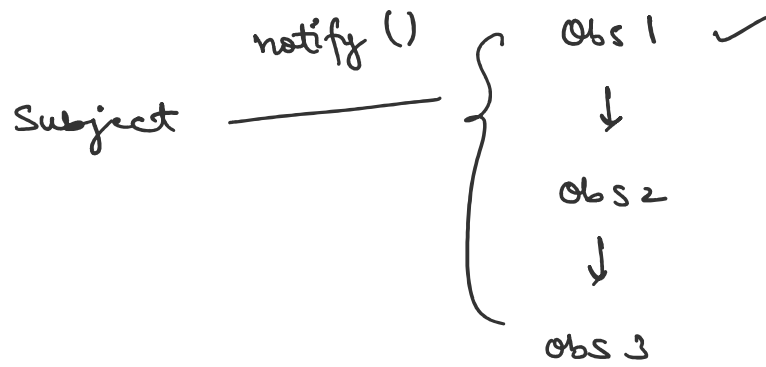


Publish - Subscribe | Pub-Sub



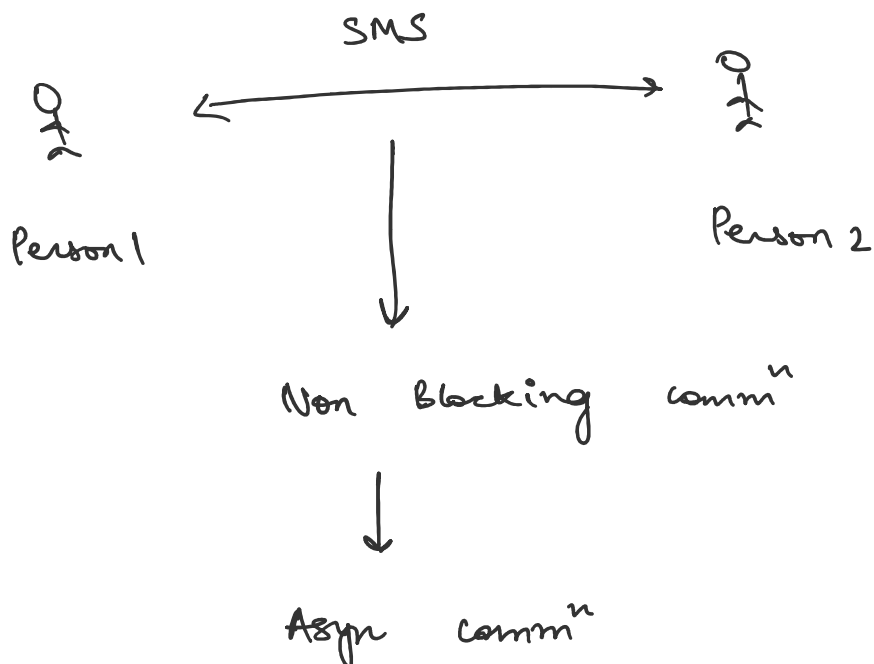
Syn vs asyn notifications

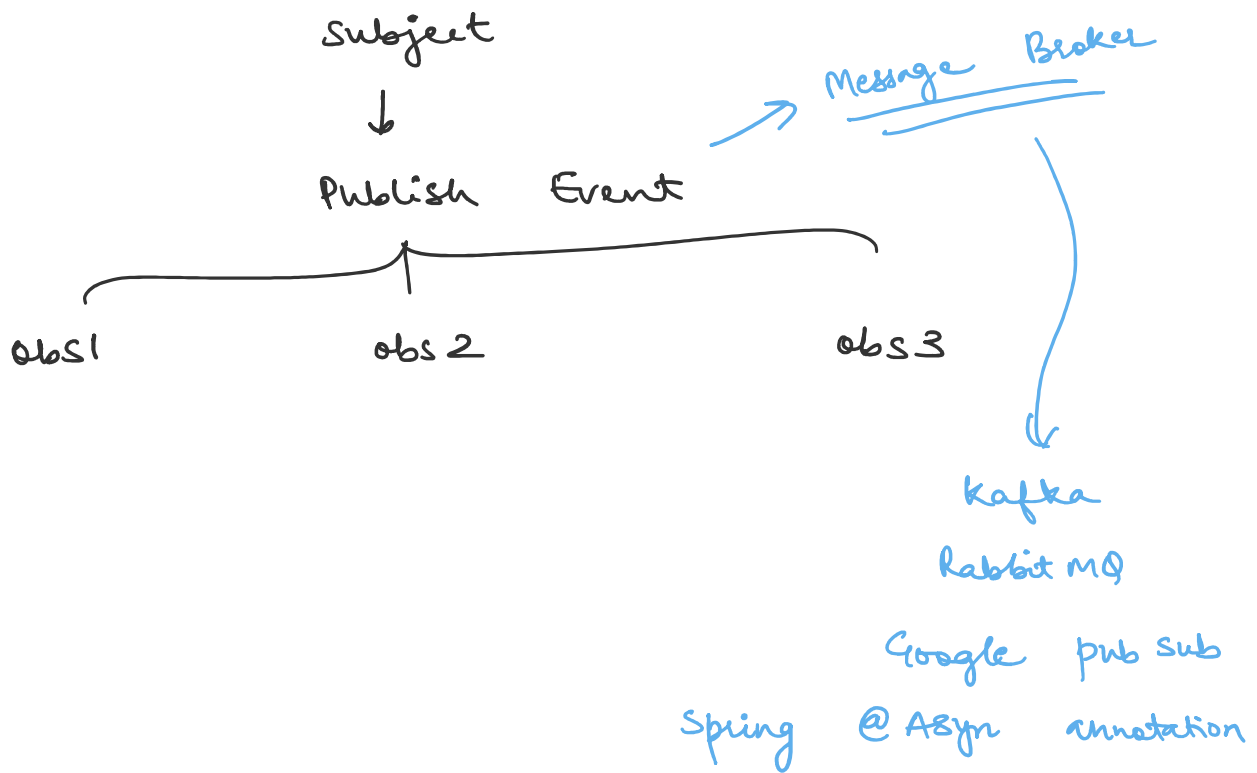




```
for (observer obs : observer List)  
{  
    obs.notify() ←  
}
```

Asyn commⁿ



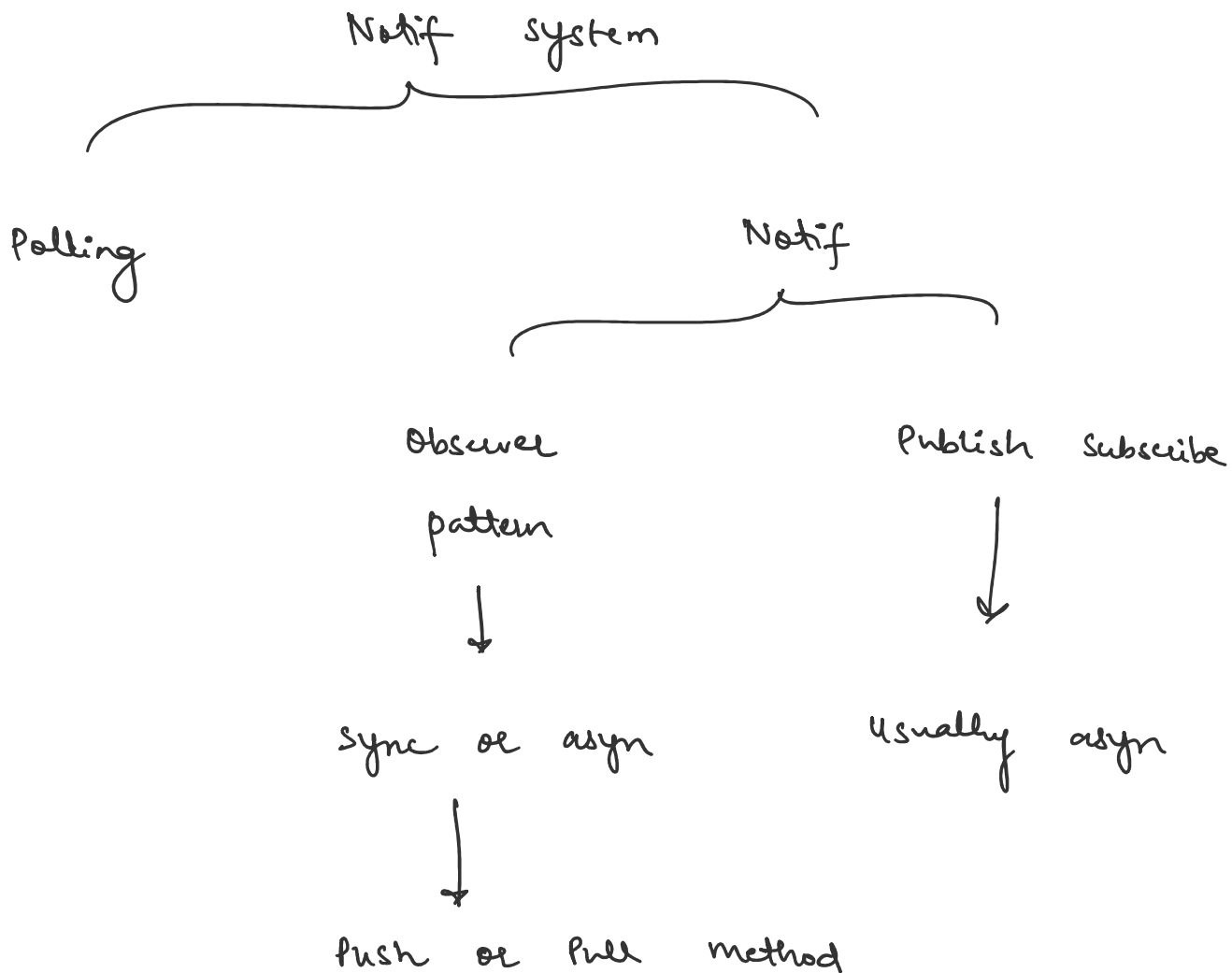


Polling vs Notification
↓ ↓
pull push

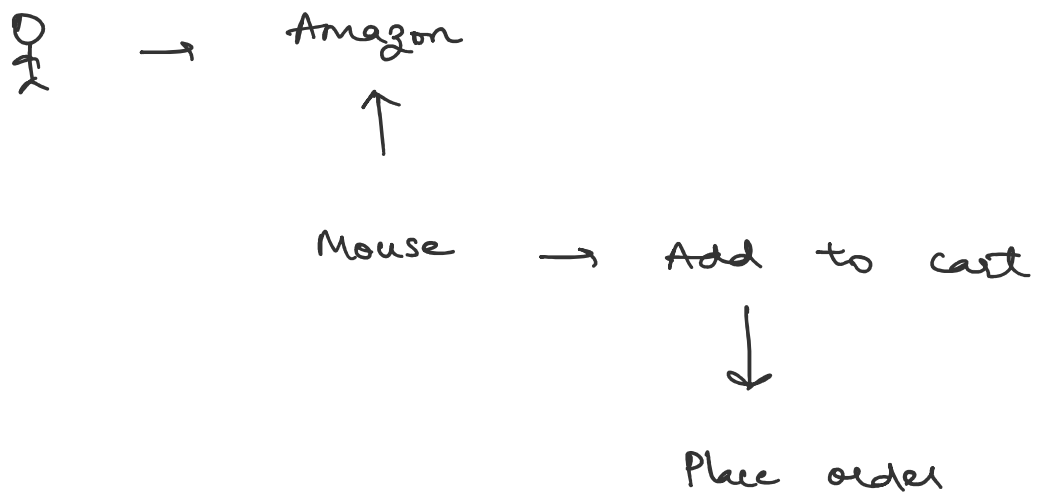
Polling (Pull)



Notification (Push)

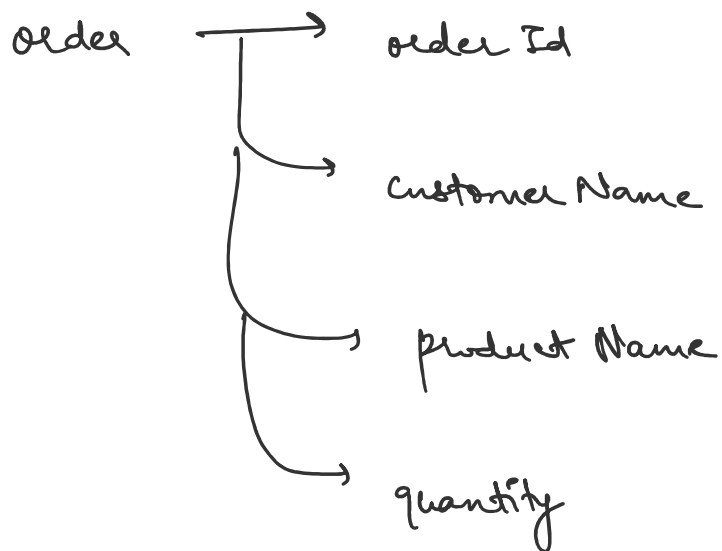
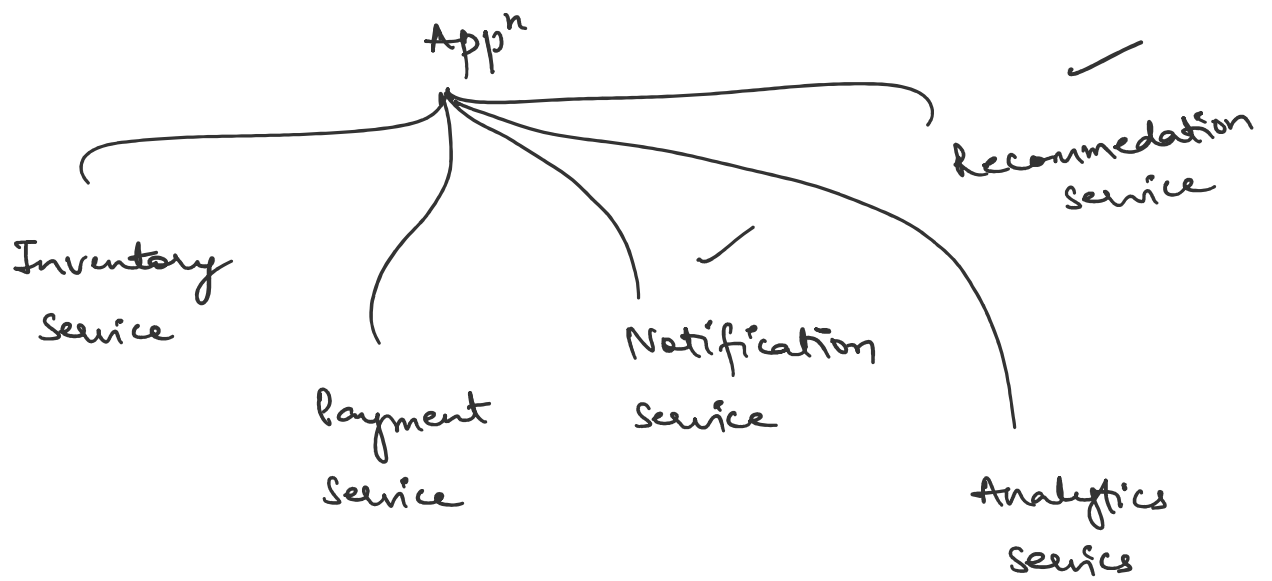


E-commerce order processing

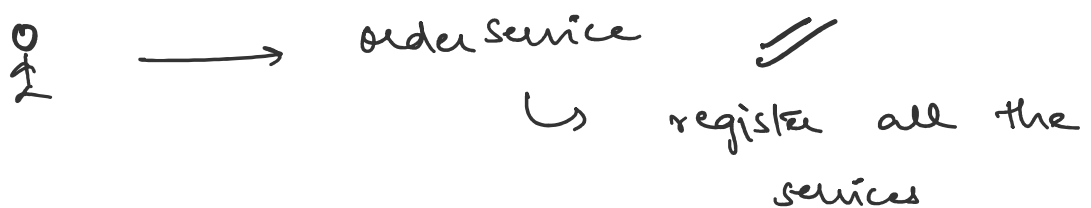
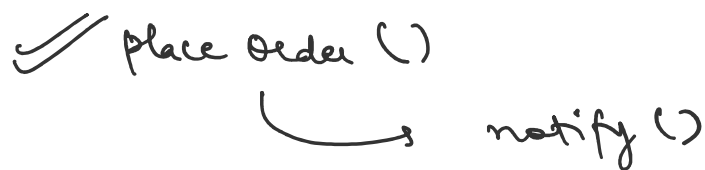
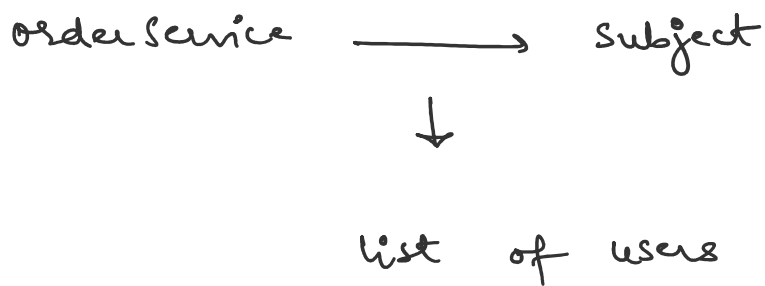
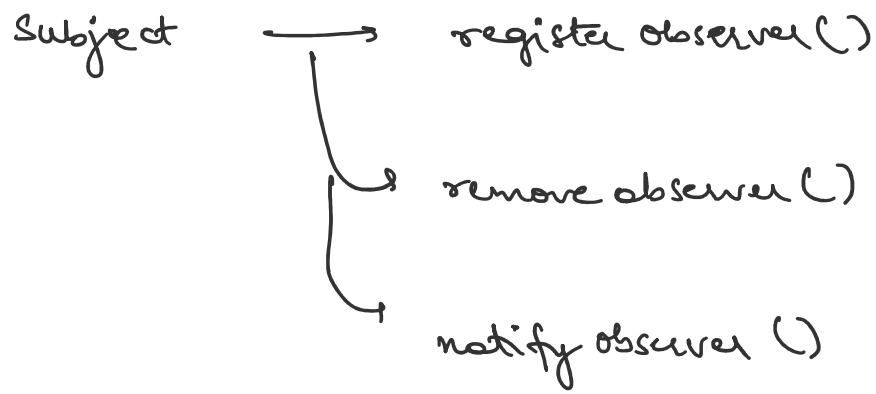


- Payment
- Reserving product
- order confirmation
- update
- Recommendation Engine //





observer → update ()



Food Delivery Apps (Swiggy / Zomato)

Imagine you're building a food delivery application like **Swiggy** or **Zomato**.

Whenever the status of an order changes (for example, **Preparing** → **Order Prepared** → **Picked Up** → **Out for Delivery**), multiple parts of the system need to react automatically.

For example:

- The **Customer App** should inform the customer about the latest order status.
- The **Delivery Partner App** should notify the delivery partner when the order is ready for pickup.
- The **Restaurant Dashboard** should display the updated order status.
- The **Notification Service** should send push notifications, SMS, or emails.
- The **Analytics Service** should update reports and delivery metrics.

We don't want the **OrderService** to call each of these services directly because that would tightly couple the system and make it difficult to add or remove services in the future.

Instead, we'll use the **Observer Pattern**, where all interested services register with the **OrderService**. Whenever the order status changes, the **OrderService** automatically notifies all registered observers, and each observer performs its own task independently.

Step 1: Order Status Enum

Create an enum to represent all possible stages of an order, from **ORDER_PLACED** to **DELIVERED**.

```
public enum OrderStatus {  
    ORDER_PLACED,  
    RESTAURANT_ACCEPTED,  
    PREPARING,  
    ORDER_PREPARED,  
    PICKED_UP,  
    OUT_FOR_DELIVERY,  
    DELIVERED  
}
```

Step 2: Order Model

Create an **Order** class to store order details such as order ID, customer name, restaurant name, and the current order status. This object will be shared with all observers whenever the status changes.

```
public class Order {  
    private final int orderId;  
    private final String customerName;  
    private final String restaurantName;  
  
    private OrderStatus status;  
  
    public Order(int orderId, String customerName, String  
restaurantName, OrderStatus status) {  
        this.orderId = orderId;  
        this.customerName = customerName;  
        this.restaurantName = restaurantName;  
        this.status = status;  
    }  
}
```

```
public int getOrderId() {  
    return orderId;  
}  
  
public String getCustomerName() {  
    return customerName;  
}  
  
public String getRestaurantName() {  
    return restaurantName;  
}  
  
public OrderStatus getStatus() {  
    return status;  
}  
  
public void setStatus(OrderStatus status) {  
    this.status = status;  
}  
}
```

Step 3: Observer Interface

Create the **Observer** interface. Every component that wants to receive order updates must implement this interface.

```
public interface Observer {  
    void update(Order order);  
}
```

Step 4: Subject Interface

Create the **Subject** interface. It defines methods to register observers,

remove observers, and notify all registered observers when the order status changes.

```
public interface Subject {  
    void registerObserver(Observer observer);  
    void removeObserver(Observer observer);  
    void notifyObservers();  
}
```

Step 5: Order Service (Concrete Subject)

Create the **OrderService**, which acts as the **Subject**. It maintains the list of observers and notifies all of them whenever the order status is updated.

```
import java.util.*;
```

```
public class OrderService implements Subject {  
    private final List<Observer> observers = new ArrayList<>();  
    private Order currentOrder;
```

```
    @Override
```

```
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
        System.out.println(observer + " subscribed.");  
    }
```

```
    @Override
```

```
    public void removeObserver(Observer observer) {  
        observers.remove(observer);  
        System.out.println(observer + " unsubscribed.");  
    }
```

```
    @Override
```

```

public void notifyObservers() {
    for (Observer observer : observers) {
        observer.update(currentOrder);
    }
}

public void updateOrderStatus(Order order, OrderStatus
status) {
    order.setStatus(status);

    currentOrder = order;

    System.out.println();
    System.out.println("-----");
    System.out.println();
    System.out.println("Order Status Updated");
    System.out.println("Order Id : " + order.getId());
    System.out.println("Customer : " +
order.getCustomerName());
    System.out.println("Restaurant : " +
order.getRestaurantName());
    System.out.println("Status : " + status);
    System.out.println();
    System.out.println("-----");
    System.out.println();

    notifyObservers();
}
}

```

Step 6: Customer App

Create the **CustomerApp**, which acts as an observer. It receives updates and informs the customer when the order is ready.

```

public class CustomerApp implements Observer {
    @Override
    public void update(Order order) {
        if (order.getStatus() == OrderStatus.ORDER_PREPARED) {
            System.out.println("--- Customer App ---");
            System.out.println("Hi " + order.getCustomerName() + ",
your order is ready.");
            System.out.println();
        }
    }

    @Override
    public String toString() {
        return "Customer App";
    }
}

```

Step 7: Delivery Partner App

Create the **DeliveryPartnerApp**, another observer. It receives updates and tells the delivery partner when the order is ready for pickup.

```

public class DeliveryPartnerApp implements Observer {
    @Override
    public void update(Order order) {
        if (order.getStatus() == OrderStatus.ORDER_PREPARED) {
            System.out.println("--- Delivery Partner ---");
            System.out.println("Restaurant has prepared the order.");
            System.out.println("Proceed to pick up Order #" +
order.getId());
            System.out.println();
        }
    }

    @Override

```

```
    public String toString() {  
        return "Delivery Partner App";  
    }  
}
```

Step 8: Restaurant Dashboard

Create the **RestaurantDashboard** observer. It displays the latest order status to the restaurant staff.

```
public class RestaurantDashboard implements Observer {  
    @Override  
    public void update(Order order) {  
        System.out.println("--- Restaurant Dashboard ---");  
        System.out.println("Current Status : " + order.getStatus());  
        System.out.println();  
    }  
  
    @Override  
    public String toString() {  
        return "Restaurant Dashboard";  
    }  
}
```

Step 9: Notification Service

Create the **NotificationService** observer. It sends notifications such as push notifications, SMS, or email whenever the order status changes.

```
public class NotificationService implements Observer {  
    @Override  
    public void update(Order order) {
```

```

        System.out.println("--- Notification Service ---");
        System.out.println("Sending Push Notification.");
        System.out.println("Sending SMS.");
        System.out.println("Sending Email.");
        System.out.println();
    }

    @Override
    public String toString() {
        return "Notification Service";
    }
}

```

Step 10: Analytics Service

Create the **AnalyticsService** observer. It updates reports, dashboards, and delivery metrics whenever the order status changes.

```

public class AnalyticsService implements Observer {
    @Override
    public void update(Order order) {
        System.out.println("--- Analytics Service ---");
        System.out.println("Updating delivery metrics.");
        System.out.println("Updating restaurant performance.");
        System.out.println("Updating live dashboard.");
        System.out.println();
    }

    @Override
    public String toString() {
        return "Analytics Service";
    }
}

```

Step 11: Client

Finally, create the client program. It creates the **OrderService**, registers all observers, updates the order status, and demonstrates how all registered observers are notified automatically.

```
public class Main {  
    public static void main(String[] args) {  
        OrderService orderService = new OrderService();  
  
        Observer customer = new CustomerApp();  
        Observer delivery = new DeliveryPartnerApp();  
        Observer restaurant = new RestaurantDashboard();  
        Observer notification = new NotificationService();  
        Observer analytics = new AnalyticsService();  
  
        orderService.registerObserver(customer);  
        orderService.registerObserver(delivery);  
        orderService.registerObserver(restaurant);  
        orderService.registerObserver(notification);  
        orderService.registerObserver(analytics);  
  
        Order order = new Order(1001, "Alice", "Domino's Pizza",  
                                OrderStatus.PREPARING);  
  
        orderService.updateOrderStatus(order,  
                                OrderStatus.ORDER_PREPARED);  
  
        System.out.println("-----");  
        System.out.println();  
  
        System.out.println("Analytics Service is offline.");  
  
        orderService.removeObserver(analytics);  
  
        orderService.updateOrderStatus(order,
```

```
OrderStatus.PICKED_UP);  
    }  
}
```

/ **

Output :

*Customer App subscribed.
Delivery Partner App subscribed.
Restaurant Dashboard subscribed.
Notification Service subscribed.
Analytics Service subscribed.*

*Order Status Updated
Order Id : 1001
Customer : Alice
Restaurant : Domino's Pizza
Status : ORDER_PREPARED*

*--- Customer App ---
Hi Alice, your order is ready.*

*--- Delivery Partner ---
Restaurant has prepared the order.
Proceed to pick up Order #1001*

*--- Restaurant Dashboard ---
Current Status : ORDER_PREPARED*

*--- Notification Service ---
Sending Push Notification.
Sending SMS.
Sending Email.*

*--- Analytics Service ---
Updating delivery metrics.
Updating restaurant performance.
Updating live dashboard.*

*-----
Analytics Service is offline.
Analytics Service unsubscribed.*

*-----
Order Status Updated
Order Id : 1001
Customer : Alice
Restaurant : Domino's Pizza
Status : PICKED_UP*

*-----
--- Restaurant Dashboard ---
Current Status : PICKED_UP*

*--- Notification Service ---
Sending Push Notification.
Sending SMS.
Sending Email.*

** */*

E-Commerce Order Processing

Imagine you're building an **e-commerce application** like Amazon or

Flipkart.

Whenever a **customer places an order**, multiple parts of the system need to react automatically.

For example:

- The **Inventory Service** should reserve the ordered products.
- The **Payment Service** should process the payment and generate an invoice.
- The **Notification Service** should send order confirmation messages.
- The **Analytics Service** should update sales reports and dashboards.
- The **Recommendation Service** should update product recommendations.

We don't want the **OrderService** to call each of these services directly because that would tightly couple the system and make it difficult to add or remove services later.

Instead, we'll use the **Observer Pattern**, where all interested services register with the **OrderService**. Whenever a new order is placed, the **OrderService** automatically notifies all registered observers, and each observer performs its own task independently.

Step 1: Create Order Model

```
public class Order {  
    private final int orderId;  
    private final String customerName;  
    private final String productName;  
    private final int quantity;  
  
    public Order(int orderId, String customerName, String  
productName, int quantity) {  
        this.orderId = orderId;  
    }  
}
```

```
        this.customerName = customerName;
        this.productName = productName;
        this.quantity = quantity;
    }

    public int getOrderId() {
        return orderId;
    }

    public String getCustomerName() {
        return customerName;
    }

    public String getProductName() {
        return productName;
    }

    public int getQuantity() {
        return quantity;
    }
}
```

Step 2: Create the Observer Interface

```
public interface Observer {
    void update(Order order);
}
```

Step 3: Create the Subject Interface

```
public interface Subject {
    void registerObserver(Observer observer);
}
```

```

void removeObserver(Observer observer);
void notifyObservers();
}

```

Step 4: Create OrderService (Concrete Subject)

```
import java.util.*;
```

```
public class OrderService implements Subject {
    private final List<Observer> observers = new ArrayList<>();
    private Order currentOrder;
```

```
@Override
```

```
public void registerObserver(Observer observer) {
    observers.add(observer);
    System.out.println(observer + " subscribed.");
}
```

```
@Override
```

```
public void removeObserver(Observer observer) {
    observers.remove(observer);
    System.out.println(observer + " unsubscribed.");
}
```

```
@Override
```

```
public void notifyObservers() {
    System.out.println();
    System.out.println("Notifying " + observers.size() + "
subscribed services...\n");
```

```
    for (Observer observer : observers) {
```

```
        System.out.println("-----");
    }
```

```
System.out.println("Notifying -> " + observer);
observer.update(currentOrder);
System.out.println(observer + " completed.");
```

```
System.out.println("-----\n");
    }
}
```

```
public void placeOrder(Order order) {
```

```
System.out.println("\n=====
=====");
```

```
    System.out.println("Customer clicked PLACE ORDER");
```

```
System.out.println("=====
=====");
```

```
    System.out.println("Order Id    : " + order.getId());
```

```
    System.out.println("Customer Name : " +
```

```
order.getCustomerName());
```

```
    System.out.println("Product      : " +
```

```
order.getProductName());
```

```
    System.out.println("Quantity     : " + order.getQuantity());
```

```
    currentOrder = order;
```

```
    System.out.println("\nOrder created successfully.");
    notifyObservers();
```

```
    System.out.println("All services finished processing the
order.");
```

```
System.out.println("=====
=====
=====");
```

```
    }
}
```

Step 5: Inventory Service

```
public class InventoryService implements Observer{
    @Override
    public void update(Order order){
        System.out.println("===== Inventory Service
=====");
        System.out.println("Received Order #" +
order.getId());
        System.out.println("Checking stock for " +
order.getProductName());
        System.out.println("Current Stock : 50");
        System.out.println("Requested Quantity : " +
order.getQuantity());
        System.out.println("Stock available.");
        System.out.println("Reserving inventory...");
        System.out.println("Updating warehouse database...");
        System.out.println("Inventory updated successfully.");
    }

    public String toString(){
        return "Inventory Service";
    }
}
```

Step 6: Payment Service

```
public class PaymentService implements Observer{
    @Override
    public void update(Order order){
        System.out.println("===== Payment Service
=====");
        System.out.println("Received Order #" +
```

```

order.getId();
    System.out.println("Fetching payment details...");
    System.out.println("Validating payment...");
    System.out.println("Authorizing payment...");
    System.out.println("Payment successful.");
    System.out.println("Generating invoice INV-" +
order.getId());
    System.out.println("Updating order status to PAID.");
}

public String toString(){
    return "Payment Service";
}
}

```

Step 7: Notification Service

```

public class NotificationService implements Observer {
    @Override
    public void update(Order order) {
        System.out.println("==== Notification Service
=====");
        System.out.println("Preparing confirmation message...");
        System.out.println("Sending Email to " +
order.getCustomerName());
        System.out.println("Sending SMS.");
        System.out.println("Sending Push Notification.");
        System.out.println("Customer notified successfully.");
    }

    public String toString() {
        return "Notification Service";
    }
}

```

Step 8: Analytics Service

```
public class AnalyticsService implements Observer {
    @Override
    public void update(Order order) {
        System.out.println("===== Analytics Service
=====");
        System.out.println("Updating revenue.");
        System.out.println("Incrementing order count.");
        System.out.println("Updating product sales.");
        System.out.println("Refreshing dashboards.");
        System.out.println("Analytics updated.");
    }

    public String toString() {
        return "Analytics Service";
    }
}
```

Step 9: Recommendation Service

```
public class RecommendationService implements Observer {
    @Override
    public void update(Order order) {
        System.out.println("===== Recommendation Service
=====");
        System.out.println("Analyzing purchase history...");
        System.out.println("Product purchased : " +
order.getProductName());
        System.out.println("Generating recommendations...");
        System.out.println("- Mouse Pad");
        System.out.println("- USB Hub");
        System.out.println("- Laptop Stand");
    }
}
```

```

        System.out.println("Recommendation engine updated.");
    }

    public String toString() {
        return "Recommendation Service";
    }
}

```

Step 10: Client

```

public class Main {
    public static void main(String[] args) {
        OrderService orderService = new OrderService();

        Observer inventory = new InventoryService();
        Observer payment = new PaymentService();
        Observer notification = new NotificationService();
        Observer analytics = new AnalyticsService();
        Observer recommendation = new
RecommendationService();

        orderService.registerObserver(inventory);
        orderService.registerObserver(payment);
        orderService.registerObserver(notification);
        orderService.registerObserver(analytics);
        orderService.registerObserver(recommendation);

        System.out.println();

        Order order1 = new Order(101, "Alice", "Wireless Mouse",
2);
        orderService.placeOrder(order1);

        System.out.println("Analytics Service is under maintenance.
\n");
    }
}

```

```
orderService.removeObserver(analytics);
```

```
Order order2 = new Order(102, "Bob", "Mechanical  
Keyboard", 1);  
orderService.placeOrder(order2);  
}  
}
```

```
/* *
```

Output :

Inventory Service subscribed.

Payment Service subscribed.

Notification Service subscribed.

Analytics Service subscribed.

Recommendation Service subscribed.

```
=====
```

Customer clicked PLACE ORDER

```
=====
```

Order Id : 101

Customer Name : Alice

Product : Wireless Mouse

Quantity : 2

Order created successfully.

Notifying 5 subscribed services...

Notifying -> Inventory Service

===== Inventory Service =====

Received Order #101

Checking stock for Wireless Mouse

Current Stock : 50
Requested Quantity : 2
Stock available.
Reserving inventory...
Updating warehouse database...
Inventory updated successfully.
Inventory Service completed.

Notifying -> Payment Service
===== Payment Service =====
Received Order #101
Fetching payment details...
Validating payment...
Authorizing payment...
Payment successful.
Generating invoice INV-101
Updating order status to PAID.
Payment Service completed.

Notifying -> Notification Service
===== Notification Service =====
Preparing confirmation message...
Sending Email to Alice
Sending SMS.
Sending Push Notification.
Customer notified successfully.
Notification Service completed.

Notifying -> Analytics Service
===== Analytics Service =====
Updating revenue.
Incrementing order count.
Updating product sales.

Refreshing dashboards.
Analytics updated.
Analytics Service completed.

Notifying -> Recommendation Service
===== Recommendation Service =====
Analyzing purchase history...
Product purchased : Wireless Mouse
Generating recommendations...
- Mouse Pad
- USB Hub
- Laptop Stand
Recommendation engine updated.
Recommendation Service completed.

All services finished processing the order.

=====

=====

Analytics Service is under maintenance.

Analytics Service unsubscribed.

=====

=====

Customer clicked PLACE ORDER

=====

=====

Order Id : 102
Customer Name : Bob
Product : Mechanical Keyboard
Quantity : 1

Order created successfully.

Notifying 4 subscribed services...

Notifying -> Inventory Service

===== Inventory Service =====

Received Order #102

Checking stock for Mechanical Keyboard

Current Stock : 50

Requested Quantity : 1

Stock available.

Reserving inventory...

Updating warehouse database...

Inventory updated successfully.

Inventory Service completed.

Notifying -> Payment Service

===== Payment Service =====

Received Order #102

Fetching payment details...

Validating payment...

Authorizing payment...

Payment successful.

Generating invoice INV-102

Updating order status to PAID.

Payment Service completed.

Notifying -> Notification Service

===== Notification Service =====

Preparing confirmation message...

Sending Email to Bob

Sending SMS.

Sending Push Notification.

Customer notified successfully.

Notification Service completed.

Notifying -> Recommendation Service

===== Recommendation Service =====

Analyzing purchase history...

Product purchased : Mechanical Keyboard

Generating recommendations...

- Mouse Pad

- USB Hub

- Laptop Stand

Recommendation engine updated.

Recommendation Service completed.

All services finished processing the order.

=====

* */

Ride Booking (Uber/Ola)

Imagine you're building a ride booking application like **Uber** or **Ola**.

Whenever the **ride status changes** (for example, **Accepted → Driver Arriving → Trip Started → Trip Completed**), multiple parts of the system need to react automatically.

For example:

- The **Passenger App** should show the latest ride status.

- The **Driver App** should display the next action for the driver.
- The **Maps Service** should update navigation and ETA.
- The **Notification Service** should send push notifications, SMS, or emails.
- The **Pricing Service** should calculate the final fare when the trip is completed.
- The **Analytics Service** should update ride statistics and business dashboards.

We don't want the **RideService** to call each of these services directly because that would tightly couple the system and make it difficult to add or remove services later.

Instead, we'll use the **Observer Pattern**, where all interested services register with the **RideService**. Whenever the ride status changes, the **RideService** automatically notifies all registered observers, and each observer performs its own task independently.

Step 1: Ride Status Enum

Create an enum to represent the different stages of a ride.

```
public enum RideStatus {  
    REQUESTED,  
    ACCEPTED,  
    DRIVER_ARRIVING,  
    TRIP_STARTED,  
    TRIP_COMPLETED,  
    CANCELLED  
}
```

Step 2: Ride Model

Create a Ride class to store ride details and the current ride status.

```
public class Ride {
    private final int rideId;
    private final String passengerName;
    private final String driverName;
    private final String pickupLocation;
    private final String destination;

    private RideStatus status;

    public Ride(int rideId, String passengerName, String
driverName, String pickupLocation, String destination,
RideStatus status) {
        this.rideId = rideId;
        this.passengerName = passengerName;
        this.driverName = driverName;
        this.pickupLocation = pickupLocation;
        this.destination = destination;
        this.status = status;
    }

    public int getRideId() {
        return rideId;
    }

    public String getPassengerName() {
        return passengerName;
    }

    public String getDriverName() {
        return driverName;
    }

    public String getPickupLocation() {
        return pickupLocation;
    }
}
```

```
public String getDestination() {  
    return destination;  
}  
  
public RideStatus getStatus() {  
    return status;  
}  
  
public void setStatus(RideStatus status) {  
    this.status = status;  
}  
}
```

Step 3: Observer Interface

Create the Observer interface. Every component that wants to receive ride updates must implement this interface.

```
public interface Observer {  
    void update(Ride ride);  
}
```

Step 4: Subject Interface

Create the Subject interface. It manages the registration, removal, and notification of observers.

```
public interface Subject {  
    void registerObserver(Observer observer);  
    void removeObserver(Observer observer);  
    void notifyObservers();  
}
```

```
}
```

Step 5: Ride Service (Concrete Subject)

Create the RideService, which acts as the **Subject**. It maintains the list of observers and notifies them whenever the ride status changes.

```
import java.util.*;
```

```
public class RideService implements Subject {  
    private final List<Observer> observers = new ArrayList<>();  
    private Ride currentRide;
```

```
    @Override
```

```
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
        System.out.println(observer + " subscribed.");  
    }
```

```
    @Override
```

```
    public void removeObserver(Observer observer) {  
        observers.remove(observer);  
        System.out.println(observer + " unsubscribed.");  
    }
```

```
    @Override
```

```
    public void notifyObservers() {  
        for (Observer observer : observers) {  
            observer.update(currentRide);  
        }  
    }
```

```
    public void updateRideStatus(Ride ride, RideStatus status) {  
        ride.setStatus(status);  
    }
```

```

        currentRide = ride;

        System.out.println();
        System.out.println("-----");
        System.out.println("Ride Status Updated");
        System.out.println("Ride Id : " + ride.getRideId());
        System.out.println("Passenger : " +
ride.getPassengerName());
        System.out.println("Driver : " + ride.getDriverName());
        System.out.println("Status : " + status);
        System.out.println("-----");
        System.out.println();

        notifyObservers();
    }
}

```

Step 6: Passenger App

Create different observer classes. Each observer receives ride updates and performs its own task.

```

public class PassengerApp implements Observer {
    @Override
    public void update(Ride ride) {
        System.out.println("--- Passenger App ---");

        switch (ride.getStatus()) {
            case ACCEPTED -> System.out.println("Your driver " +
ride.getDriverName() + " has accepted your ride.");

            case DRIVER_ARRIVING -> System.out.println("Driver is
arriving.");

            case TRIP_STARTED -> System.out.println("Enjoy your

```

```

ride!");

        case TRIP_COMPLETED -> System.out.println("Trip
completed successfully.");

        default -> System.out.println("Invalid ride status");
    }

    System.out.println();
}

@Override
public String toString() {
    return "Passenger App";
}
}

```

Step 7: Driver App

```

public class DriverApp implements Observer {
    @Override
    public void update(Ride ride) {
        System.out.println("--- Driver App ---");

        switch (ride.getStatus()) {
            case ACCEPTED -> System.out.println("Proceed to " +
ride.getPickupLocation());

            case TRIP_STARTED -> System.out.println("Navigate to
" + ride.getDestination());

            case TRIP_COMPLETED -> System.out.println("Ride
completed.");

```

```

        default -> System.out.println("Ride cancelled.");
    }

    System.out.println();
}

@Override
public String toString() {
    return "Driver App";
}
}

```

Step 8: Pricing Service

```

public class PricingService implements Observer {
    @Override
    public void update(Ride ride) {
        System.out.println("--- Pricing Service ---");

        if (ride.getStatus() == RideStatus.TRIP_COMPLETED) {
            System.out.println("Calculating final fare.");
            System.out.println("Generating trip receipt.");
        } else {
            System.out.println("Pricing remains unchanged.");
        }
        System.out.println();
    }

    @Override
    public String toString() {
        return "Pricing Service";
    }
}

```

Step 9: Notification Service

```
public class NotificationService implements Observer {  
    @Override  
    public void update(Ride ride) {  
        System.out.println("--- Notification Service ---");  
        System.out.println("Sending Push Notification.");  
        System.out.println("Sending SMS.");  
        System.out.println("Sending Email.");  
        System.out.println();  
    }  
  
    @Override  
    public String toString() {  
        return "Notification Service";  
    }  
}
```

Step 10: Maps Service

```
public class MapsService implements Observer {  
    @Override  
    public void update(Ride ride) {  
        System.out.println("--- Maps Service ---");  
  
        switch (ride.getStatus()) {  
            case ACCEPTED -> System.out.println("Displaying  
driver's live location.");  
  
            case DRIVER_ARRIVING ->
```

```

System.out.println("Updating ETA.");

        case TRIP_STARTED -> System.out.println("Starting
navigation.");

        case TRIP_COMPLETED ->
System.out.println("Stopping navigation.");

        default -> System.out.println("Unknown ride status");
    }

    System.out.println();
}

@Override
public String toString() {
    return "Maps Service";
}
}

```

Step 11: Analytics Service

```

public class AnalyticsService implements Observer {
    @Override
    public void update(Ride ride) {
        System.out.println("--- Analytics Service ---");
        System.out.println("Updating ride metrics.");
        System.out.println("Updating driver statistics.");
        System.out.println("Updating business dashboard.");
        System.out.println();
    }

    @Override
    public String toString() {

```

```
        return "Analytics Service";
    }
}
```

Step 12: Client

Create the client program to register observers, update the ride status, and demonstrate how all observers receive notifications automatically.

```
public class Main {
    public static void main(String[] args) {
        RideService rideService = new RideService();

        Observer passenger = new PassengerApp();
        Observer driver = new DriverApp();
        Observer pricing = new PricingService();
        Observer notification = new NotificationService();
        Observer maps = new MapsService();
        Observer analytics = new AnalyticsService();

        rideService.registerObserver(passenger);
        rideService.registerObserver(driver);
        rideService.registerObserver(pricing);
        rideService.registerObserver(notification);
        rideService.registerObserver(maps);
        rideService.registerObserver(analytics);

        Ride ride = new Ride(501, "Alice", "John", "Airport", "City
Mall", RideStatus.REQUESTED);

        rideService.updateRideStatus(ride,
RideStatus.ACCEPTED);

        System.out.println("-----");
    }
}
```

```

        rideService.updateRideStatus(ride,
RideStatus.TRIP_STARTED);

        System.out.println("-----");

        System.out.println("Analytics Service is offline.");

        rideService.removeObserver(analytics);

        rideService.updateRideStatus(ride,
RideStatus.TRIP_COMPLETED);
    }
}

/* *

```

Output :

*Passenger App subscribed.
Driver App subscribed.
Pricing Service subscribed.
Notification Service subscribed.
Maps Service subscribed.
Analytics Service subscribed.*

*-----
Ride Status Updated
Ride Id : 501
Passenger : Alice
Driver : John
Status : ACCEPTED
-----*

*--- Passenger App ---
Your driver John has accepted your ride.*

*--- Driver App ---
Proceed to Airport*

--- Pricing Service ---
Pricing remains unchanged.

--- Notification Service ---
Sending Push Notification.
Sending SMS.
Sending Email.

--- Maps Service ---
Displaying driver's live location.

--- Analytics Service ---
Updating ride metrics.
Updating driver statistics.
Updating business dashboard.

Ride Status Updated
Ride Id : 501
Passenger : Alice
Driver : John
Status : TRIP_STARTED

--- Passenger App ---
Enjoy your ride!

--- Driver App ---
Navigate to City Mall

--- Pricing Service ---
Pricing remains unchanged.

--- Notification Service ---
Sending Push Notification.
Sending SMS.

Sending Email.

*--- Maps Service ---
Starting navigation.*

*--- Analytics Service ---
Updating ride metrics.
Updating driver statistics.
Updating business dashboard.*

*Analytics Service is offline.
Analytics Service unsubscribed.*

*Ride Status Updated
Ride Id : 501
Passenger : Alice
Driver : John
Status : TRIP_COMPLETED*

*--- Passenger App ---
Trip completed successfully.*

*--- Driver App ---
Ride completed.*

*--- Pricing Service ---
Calculating final fare.
Generating trip receipt.*

*--- Notification Service ---
Sending Push Notification.
Sending SMS.
Sending Email.*

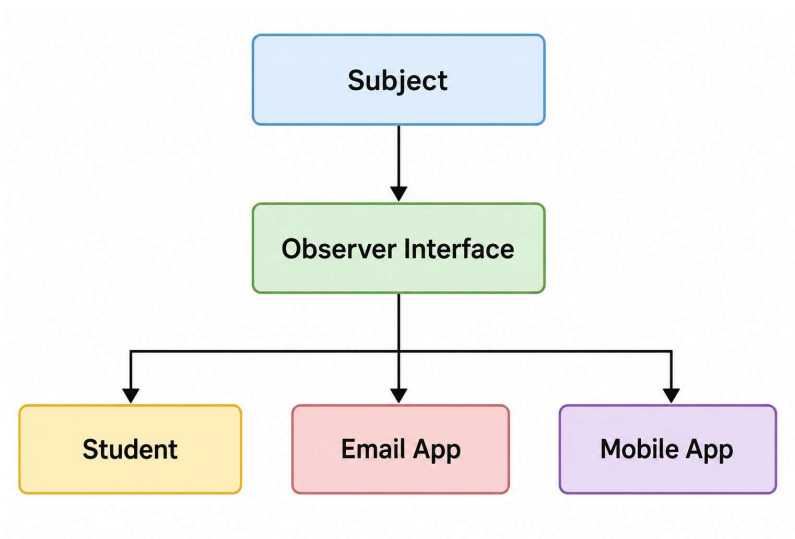
*--- Maps Service ---
Stopping navigation.*

Advantages of the Observer Pattern

The Observer Pattern promotes loose coupling, flexibility, and event-driven communication.

1. Loose Coupling

The subject communicates only with the Observer interface and does not know how observers process notifications.

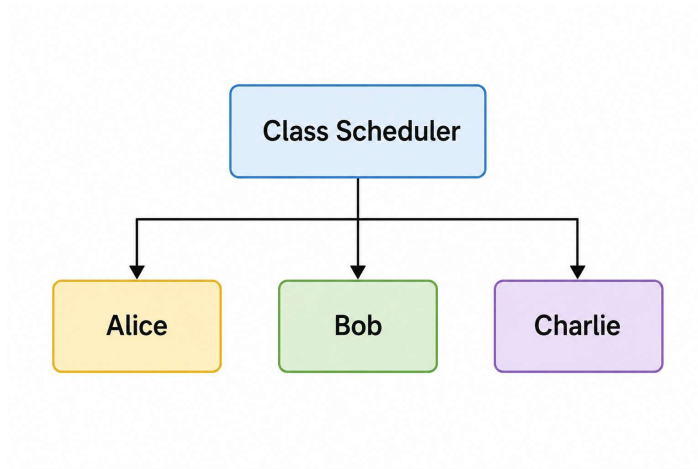


Benefits:

- Independent components
- Easier maintenance
- Better extensibility

2. One-to-Many Communication

A single subject can notify multiple observers whenever its state changes.



Common use cases:

- Notification systems
- Chat applications
- Stock market updates
- Weather monitoring
- Order processing

3. Open/Closed Principle (OCP)

New observers can be added without modifying the subject.

`class Student implements Observer`

`class EmailService implements Observer`

`class SMSService implements Observer`

The subject remains unchanged.

4. Dynamic Subscription

Observers can subscribe or unsubscribe at runtime.

```
scheduler.registerObserver(student);
```

```
scheduler.removeObserver(student);
```

5. Event-Driven Communication

Observers react automatically when an event occurs instead of continuously polling for changes.

Examples:

- Button clicked
- Order placed
- Payment successful
- Stock price updated

6. Easy to Extend

New functionality can be added simply by creating another observer.

```
class EmailNotificationService implements Observer
```

No changes are required in the subject.

Disadvantages of the Observer Pattern

Although powerful, the Observer Pattern has some limitations.

1. Notification Overhead

Whenever the subject changes, every registered observer is notified.

State Changed



Notify Observers



10,000 Observers

This can increase CPU usage and response time.

2. Performance

The subject must notify every observer.

```
for (Observer observer : observers) {  
    observer.update();  
}
```

Time Complexity:

Notify Observers = $O(n)$

where **n** is the number of observers.

3. Difficult Debugging

Observers may trigger additional events, making the execution flow difficult to trace.

Subject



Observer A



Observer B



Observer C

4. Notification Order

Observers may not receive notifications in a predictable order unless explicitly managed.

5. Memory Leaks

If observers are not removed, the subject continues to hold references to them.

```
scheduler.registerObserver(student);
```

```
// Missing  
scheduler.removeObserver(student);
```

6. Exception Handling

If one observer throws an exception, the remaining observers may not receive notifications.

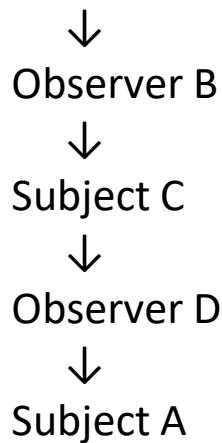
A better approach is:

```
for (Observer observer : observers) {  
    try {  
        observer.update();  
    } catch (Exception ex) {  
        // Log exception  
    }  
}
```

7. Circular Updates

Observers updating other subjects can create infinite notification loops.

Subject A



Avoid circular dependencies between subjects and observers.

When Should You Use the Observer Pattern?

Use the Observer Pattern when:

- One object's state changes and multiple objects need to react automatically.
- You want to keep the subject and observers loosely coupled.
- Observers need to subscribe or unsubscribe dynamically.
- Multiple independent components need to respond to the same event.
- Your application follows an event-driven architecture.

When Should You Avoid the Observer Pattern?

Avoid the Observer Pattern when:

- There is only one dependent object.
- Simple method calls are sufficient.
- Notification order must always be strictly controlled.
- A very large number of observers causes performance issues.

- The communication is not event-driven.

Observer Pattern vs Publish-Subscribe (Pub-Sub)

Many people think these are the same, but they are different and are used in different scenarios.

Observer Pattern

- Subject knows all its observers.
- Observers register directly with the subject.
- Notifications are sent directly.
- Mostly used inside a single application.

Examples

- YouTube subscribers
- GUI event listeners
- Spring Application Events
- Notification system within an application

Publish-Subscribe Pattern

- Publisher does **not** know its subscribers.
- Messages are sent through a **Message Broker**.
- Publishers send messages to the broker.
- The broker delivers messages to all interested subscribers.
- Commonly used in distributed systems.

Examples

- Kafka

- RabbitMQ
- Google Pub/Sub
- AWS SNS

Observer vs Publish-Subscribe

Observer Pattern

This is an Object-Oriented Design Pattern

Subject knows observers

Direct communication

Loosely coupled through interfaces

Used inside one application

Usually synchronous

Publish-Subscribe

This is an Architectural Messaging Pattern

Publisher does not know subscribers

Uses a message broker

Completely decoupled

Used in distributed systems

Usually asynchronous

Synchronous vs Asynchronous Notifications

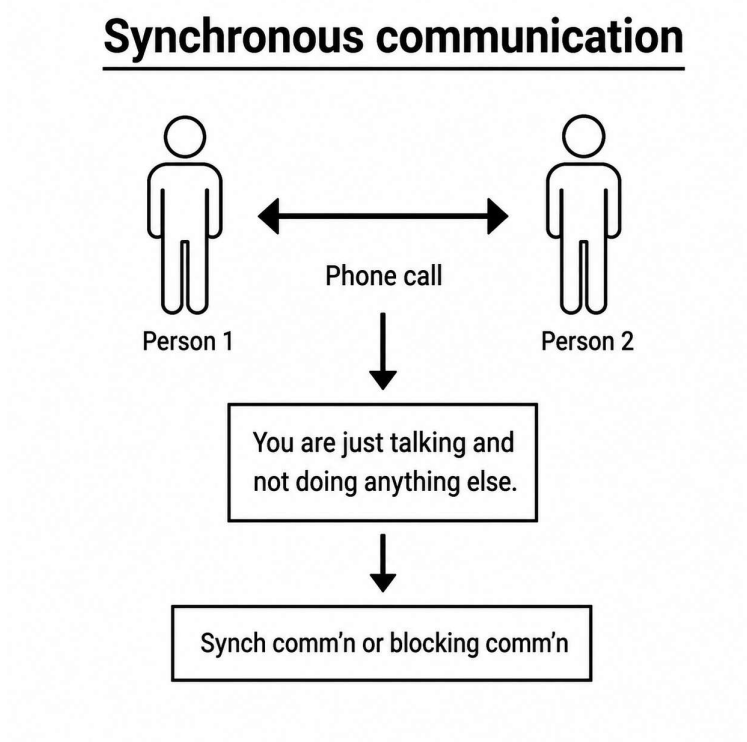
The Observer Pattern defines **how objects communicate**, but it does **not** specify **when** notifications are delivered.

Notifications can be implemented in two ways:

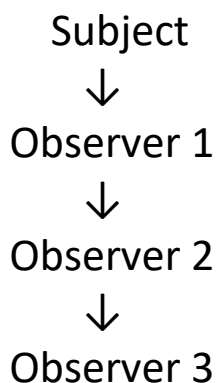
- **Synchronous**
- **Asynchronous**

The choice depends on the application's requirements.

1. Synchronous Notification



In a synchronous implementation, the **Subject** directly calls the `update()` method of each registered observer and waits for it to finish before notifying the next observer.



Example

```
for (Observer observer : observers) {  
    observer.update();  
}
```

}

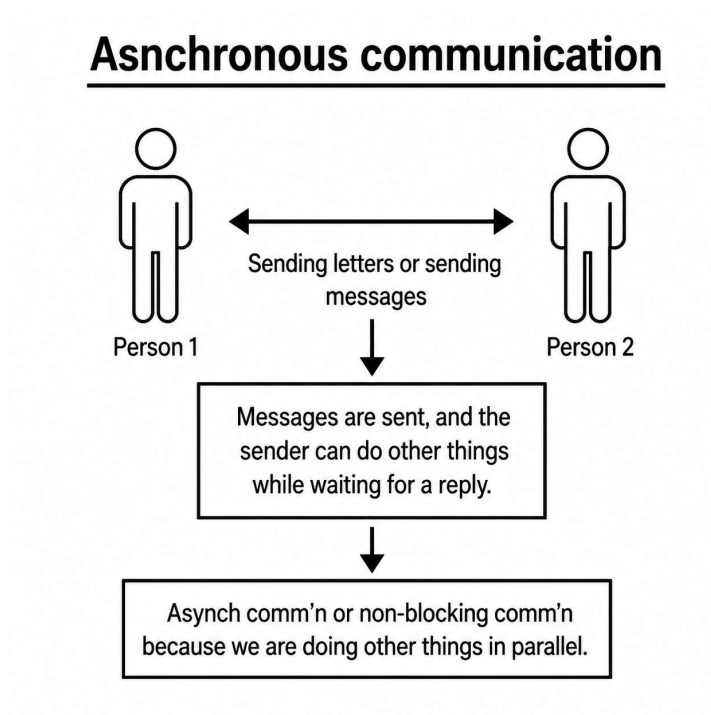
Characteristics

- Simple to implement.
- Observers execute one after another.
- The Subject waits for all observers to complete.
- Suitable for desktop and GUI applications.

Examples

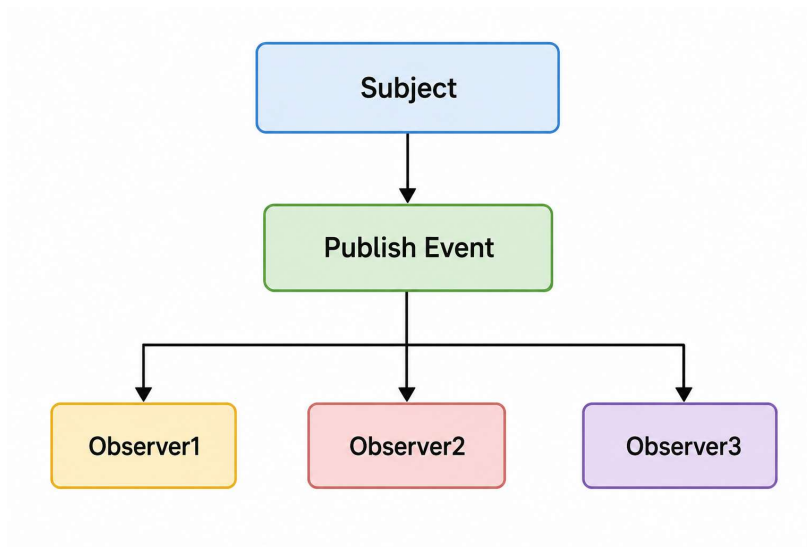
- Java Swing Event Listeners
- JavaFX Event Handlers
- Spring Application Events (default)

2. Asynchronous Notification

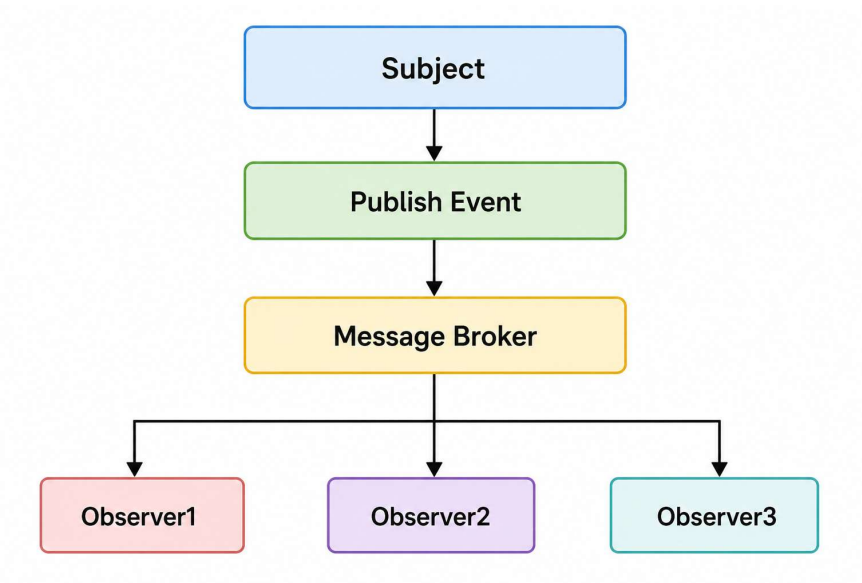


In an asynchronous implementation, the **Subject** publishes an event and continues its work without waiting for observers.

Observers process the event independently, often using separate threads or messaging systems.



In distributed systems, a message broker may deliver events to observers.



Characteristics

- The Subject does not wait for observers.
- Better scalability and throughput.
- Suitable for distributed and event-driven systems.
- Long-running tasks do not block the Subject.

Examples

- Kafka
- RabbitMQ
- Amazon SNS
- Google Pub/Sub
- Spring @Async Event Listeners

Synchronous vs Asynchronous

Synchronous

Subject waits for observers
Sequential execution
Simpler implementation
Lower scalability
Best for desktop and GUI applications

Asynchronous

Subject continues immediately
Independent execution
More complex implementation
Higher scalability
Best for distributed systems and microservices

Polling (Pull) vs Notifications (Push)

Before understanding the Observer Pattern, it's useful to compare **Polling** and **Notifications**.

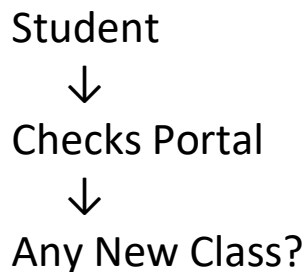
1. Polling (Pull)

In the polling model, clients repeatedly check whether new

information is available.

Example:

Students manually visit the course portal every morning to see whether a new class has been scheduled.



Problems

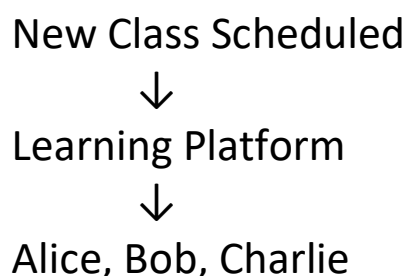
- Unnecessary requests
- Wastes resources
- Not real-time
- Users may miss updates

2. Notifications (Push)

In the push model, the system automatically notifies users whenever new information becomes available.

Example:

When a new class is scheduled, the learning platform immediately sends notifications to all subscribed students.



Benefits

- Real-time updates
- No manual checking
- More efficient
- Better user experience

The Observer Pattern follows the **notification (push)** approach, where observers are automatically informed whenever the subject's state changes.

Push Model vs Pull Model in the Observer Pattern

The Observer Pattern supports two ways of sharing data with observers after a notification is sent.

1. Pull Model

The Subject simply informs observers that **something has changed**. Observers then retrieve the data they need from the Subject.

Example Method

```
void update(Subject subject);
```

The observer calls methods on the subject to retrieve the required information.

Characteristics

- Smaller notifications
- Observer decides what data to retrieve

- Requires an additional request

2. Push Model

The Subject sends the required data along with the notification. Observers can process the notification immediately.

Example Method

```
void update(String classTitle);
```

Characteristics

- No additional request required
- Faster updates
- Easier to implement
- May send more data than some observers need

Pull Model vs Push Model

Pull Model

Subject only notifies that something changed

Observer requests data

Smaller notifications

Observer decides what to retrieve

Extra communication required

Suitable when observers need different data

Push Model

Subject sends the required data

Observer immediately processes the data

Larger notifications

Subject decides what to send

No extra communication

Suitable when all observers need the same data